27-10-2026

CO2 -ASSSESMENT TOOL 3

ERROR ANALYSIS

COURSE NAME: NATURAL LANGUAGE PROCESSING

COURSECODE: DSA0308

NAME: R SAI HASNIKA **REG NO:192424178**

Question 1: Morphological Analysis Error in Biomedical Dataset

1.1 Introduction

Morphological analysis identifies the root word, prefixes, and suffixes. In biomedical search engines, incorrect stemming can remove important information and reduce the accuracy of clinical document retrieval.

The Porter Stemmer removes word endings using predefined rules. However, it does not always preserve meaningful relationships between biomedical terms such as infection, infectious, infected, and infect.

1.2 Problem Statement

Directly applying Porter Stemming to biomedical words may produce inconsistent stems. This can cause the search engine to miss relevant clinical reports or group unrelated terms incorrectly.

1.3 Dataset Used

PubMed 20k RCT Dataset contains biomedical research abstracts and is useful for evaluating text preprocessing and medical information retrieval.

1.4 Preprocessing Steps

1. Load the biomedical text dataset.
2. Convert text to lowercase.
3. Tokenize the text into individual words.
4. Remove unnecessary punctuation.
5. Apply Porter Stemmer to each token.
6. Compare the original words with their generated stems.
7. Identify cases where meaningful information is lost.

1.5 Python Program

co2at31.py > ...

```python
import re
import pandas as pd
from nltk.stem import PorterStemmer

ps = PorterStemmer()

words = [
    "infection", "infectious", "infected", "infect",
    "infections", "infecting"
]

results = []

for word in words:
    stem = ps.stem(word)
    results.append({
        "Original Word": word,
        "Stemmed Word": stem
    })

df = pd.DataFrame(results)
print(df.to_string(index=False))
```

Expected output

```
PS C:\Users\Hasnika\OneDrive\Desktop\NLP.py> & C:\Users\Hasnika\AppData\Local\Programs\Python\Python313\python.exe c:/Users/Hasnika/OneDrive/Deskto
    infection      infect
   infectious     infecti
     infected      infect
       infect      infect
   infections      infect
    infecting      infect
PS C:\Users\Hasnika\OneDrive\Desktop\NLP.py>
```

1.6 Error Analysis

- Infection → infect: The suffix -ion forms a noun from a verb. Removing it loses the noun form.
- Infectious → infecti: The generated stem may not be a valid English word and loses the adjective-forming suffix -ous.
- Infected → infect: The suffix -ed indicates a grammatical form of the verb.
- Infecting → infect: The suffix -ing represents a grammatical form of the verb.

1.7 Improved Approach

Use tokenization, biomedical vocabulary, and lemmatization instead of relying only on stemming. Preserve meaningful terms such as infection and infectious when they represent different clinical concepts. A domain-specific dictionary can help map related terms without losing semantic information.

1.8Conclusion

Porter Stemming is useful for reducing word variations, but it may remove derivational information. A biomedical search engine should combine morphological rules, lemmatization, and domain-specific terminology to improve retrieval accuracy.

Question 2: Error Analysis of a Finite-State Morphological Parser in Social Media Data

2.1 Introduction

A Finite-State Transducer (FST) is a computational model that maps an input word to its morphological structure. It uses states and transitions to recognize prefixes, roots, and suffixes.

Social media words frequently contain multiple affixes, so a simple parser may fail to identify their complete structure.

2.2 Problem Statement

The existing parser handles standard English words but fails to recognize combinations of prefixes and suffixes in words such as happiest, unbelievable, running, reordering, smartphones, and unreadable.

2.3 Analysis of Given Words

Word	Root	Affix structure	Morphological type
Happiest	happy	-iest (spelling change: y → i)	Inflectional
Unbelievable	believe	un-, -able	Derivational
Running	run	-ing (doubled consonant)	Inflectional
Reordering	order	re-, -ing	Mixed: derivational + inflectional
Smartphones	smartphone	-s	Inflectional
Unreadable	read	un-, -able	Derivational

2.4 Why the Parser Fails

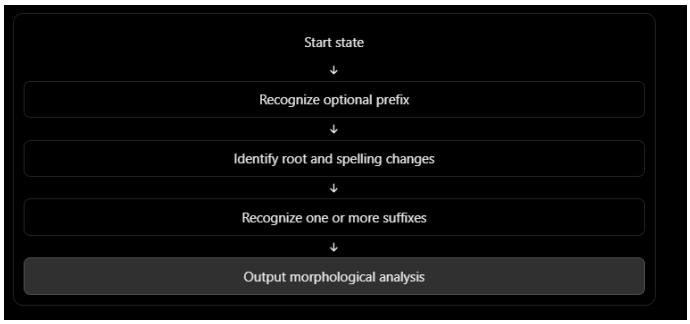
1. It recognizes only one affix instead of multiple affixes.
2. It does not account for spelling changes such as happy → happiest.
3. It does not handle consonant doubling, as in run → running.
4. It may not distinguish roots from complete words.
5. It lacks rules for irregular forms and word-specific exceptions.

2.5 Modified Python Program

```python
rules = {
    "happiest": ("happy", "", "-est", "Inflectional"),
    "unbelievable": (
        "believe", "un-", "-able", "Derivational"
    ),
    "running": ("run", "", "-ing", "Inflectional"),
    "reordering": (
        "order", "re-", "-ing",
        "Derivational + Inflectional"
    ),
    "smartphones": (
        "smartphone", "", "-s", "Inflectional"
    ),
    "unreadable": (
        "read", "un-", "-able", "Derivational"
    )
}
words = [
    "happiest", "unbelievable", "running",
    "reordering", "smartphones", "unreadable"
]
print("Word | Prefix | Root | Suffix | Type")
print("-" * 75)
for word in words:
    root, prefix, suffix, category = rules[word]
    print(
        word, "|", prefix or "None", "|",
        root, "|", suffix or "None", "|", category
    )

```

2.6 Modified FST Transitions



2.7 Accuracy Evaluation

Accuracy is calculated as:

$$\text{Accuracy} = \frac{\text{Correct analyses}}{\text{Total words}} \times 100$$

2.8 Computational Complexity

If a parser scans a word of length \(n\) using a fixed number of transition rules, its basic processing time is generally \(O(n)\). Adding more states and rules increases implementation complexity. Backtracking or repeatedly checking many rules may increase processing time.

2.9 Conclusion

A multi-stage FST with optional prefixes, root recognition, spelling-change rules, and multiple suffix transitions can improve morphological analysis of social media text. Real-world evaluation should use a labeled dataset such as Sentiment140 and measure accuracy against manually verified analyses.

Question 3: Error Analysis of Stemming in News Article Classification

3.1 Introduction

News classification systems use text features to categorize articles into topics such as technology, business, sports, and politics. Stemming reduces word variations, but incorrect stemming can remove useful information and affect classification accuracy.

3.2 Problem Statement

Words such as organization, organizer, organizing, organized, and organization's may be reduced inconsistently. This can cause the classifier to lose useful distinctions and incorrectly classify technology articles as business articles.

3.3 Dataset Used

The BBC News Dataset or AG News Dataset can be used to compare preprocessing strategies. Each dataset contains news text that can be processed and classified using machine learning.

3.4 Stemming Analysis

Original word	Possible Porter stem	Morphological form
Organization	organ	Derivational
Organizer	organ	Derivational
Organizing	organ	Inflectional verb form
Organized	organ	Inflectional verb form
Organization's	organization	Possessive form before stemming

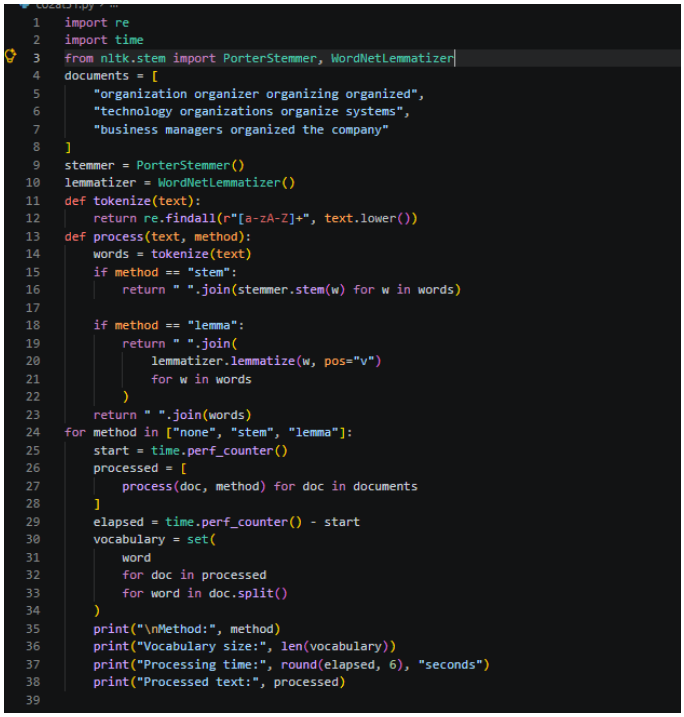
Porter stemming may collapse several of these words to organ, which is not an appropriate semantic representation of organization. The possessive form may also behave differently depending on how punctuation is tokenized.

3.5 Effect on Feature Extraction

- Stemming reduces the number of unique features.
- Incorrect stems can combine words with different meanings.

- Useful technical terms may lose their identity.
- The classifier may give excessive importance to misleading shared features.
- Lemmatization can preserve more meaningful dictionary forms.

3.6 Python Program for Comparison



```python
import re
import time
from nltk.stem import PorterStemmer, WordNetLemmatizer
documents = [
    "organization organizer organizing organized",
    "technology organizations organize systems",
    "business managers organized the company"
]
stemmer = PorterStemmer()
lemmatizer = WordNetLemmatizer()
def tokenize(text):
    return re.findall(r"[a-zA-Z]+", text.lower())
def process(text, method):
    words = tokenize(text)
    if method == "stem":
        return " ".join(stemmer.stem(w) for w in words)

    if method == "lemma":
        return " ".join(
            lemmatizer.lemmatize(w, pos="v")
            for w in words
        )
    return " ".join(words)
for method in ["none", "stem", "lemma"]:
    start = time.perf_counter()
    processed = [
        process(doc, method) for doc in documents
    ]
    elapsed = time.perf_counter() - start
    vocabulary = set(
        word
        for doc in processed
        for word in doc.split()
    )
    print("\nMethod:", method)
    print("Vocabulary size:", len(vocabulary))
    print("Processing time:", round(elapsed, 6), "seconds")
    print("Processed text:", processed)

```

3.7 Evaluation

The program compares vocabulary size and processing time. To compare classification accuracy, train the same classifier, such as Logistic Regression, on each version of the dataset and evaluate it using the same train/test split.

No actual BBC News or AG News dataset has been loaded in this example, so exact vocabulary sizes and accuracy values should be obtained by running the experiment on the downloaded dataset.

3.8 Best Preprocessing Strategy

Lemmatization is generally a better choice when preserving word meaning is important. However, the best method depends on the dataset and classification task. Keeping original terms can be useful when specific technical vocabulary is important.

3.9Conclusion

Stemming reduces vocabulary size but may merge semantically different words. Comparing no stemming, Porter stemming, and lemmatization helps determine which preprocessing method produces the best classification results with the least semantic information loss.

Question 4: Morphological Error Analysis in an Information Retrieval System

4.1 Introduction

An information retrieval system searches product descriptions using customer queries. Porter Stemmer removes suffixes to reduce word variations, but it may also combine words that have different meanings.

4.2 Problem Statement

The product search engine processes watches, watching, washable, washer, and washed. Incorrect suffix removal can cause irrelevant products to appear in search results.

4.3 Analysis of the Given Words

Word	Base/root	Morphological type	Possible issue
Watches	watch	Inflectional plural noun	May be confused with the verb form
Watching	watch	Inflectional verb form	Context determines the intended meaning
Washable	wash	Derivational	-able indicates something can be washed
Washer	wash	Derivational	-er may identify a device or person
Washed	wash	Inflectional verb form	Indicates a past or participial form

4.4 Stemming Errors

1. Watches: May refer to timepieces or the verb watches. The correct interpretation depends on context.
2. Watching: Usually a grammatical form of watch, but its role in a sentence may differ.
3. Washable: The suffix -able changes the meaning to “capable of being washed.” Removing it loses useful product information.
4. Washer: Refers to a washing machine, device, or person. Treating it as identical to wash may reduce search precision.
5. Washed: Usually an inflectional form and can often be normalized to wash.

4.5 Improved Morphological Preprocessing Strategy

- Convert text to lowercase and tokenize it.
- Apply lemmatization to common inflectional forms.
- Preserve derivational forms such as washable and washer when they convey product-specific meaning.
- Use product-category dictionaries and domain-specific vocabulary.

- Apply contextual matching to distinguish nouns from verbs.
- Evaluate the system using precision, recall, and F1-score.

4.6 Python Program

```python
from nltk.stem import WordNetLemmatizer
lemmatizer = WordNetLemmatizer()
words = [
    "watches", "watching", "washable",
    "washer", "washed"
]
# Treat each word as a noun or verb as appropriate.
parts_of_speech = {
    "watches": "v",
    "watching": "v",
    "washable": "a",
    "washer": "n",
    "washed": "v"
}
for word in words:
    pos = parts_of_speech[word]
    if pos == "a":
        normalized = word  # Preserve the derivational form
    else:
        normalized = lemmatizer.lemmatize(word, pos=pos)
    print(word, "->", normalized)

```

4.7 Expected Output

```
p/NLP.py/co2at31.py
watches -> watch
watching -> watch
washable -> washable
washer -> washer
washed -> wash
PS C:\Users\Hasnika\OneDrive\Desktop\NLP.py>
```

The output illustrates a strategy that normalizes grammatical variations while preserving meaningful derivational words. Correct results depend on the selected part of speech and the lexicon.

4.8 Conclusion

A product search engine should not remove all suffixes blindly. Inflectional forms can often be normalized, while derivational forms should be preserved when they distinguish product functions or categories. Combining lemmatization with domain-specific rules improves search relevance.

Question 5: Identify the Error, Rectify It, and Analyze the Output

5.1 Given Program

from nltk.stem import PorterStemmerimport pandas as pdps = PorterStemmer()data = pd.read_csv("BBCNews.csv")data["Processed"] = data["Text"].apply(ps.stem)print(data["Processed"].head())

5.2 Identification of Errors

1. Incorrect use of ps.stem(): Porter Stemmer expects a single word, but data["Text"] contains complete text documents.
2. Incorrect preprocessing: The text is not tokenized into individual words.
3. Missing cleaning: Punctuation and unnecessary symbols are not removed.
4. Missing validation: The program assumes that the file contains a column named Text.
5. Potential file error: The program fails if BBCNews.csv is missing or has a different column name.

5.3 Corrected Python Program

co2at31.py > ...

```python

import re
import pandas as pd
from nltk.stem import PorterStemmer
ps = PorterStemmer()
data = pd.read_csv("BBCNews.csv")
if "Text" not in data.columns:
    raise ValueError("Dataset must contain a 'Text' column.")
def process_text(text):
    if pd.isna(text):
        return ""
    words = re.findall(r"[a-zA-Z]+", str(text).lower())
    return " ".join(ps.stem(word) for word in words)
data["Processed"] = data["Text"].apply(process_text)
print(data[["Text", "Processed"]].head())

```

5.4 Explanation of the Correction

- read_csv() loads the dataset.
- re.findall() extracts individual alphabetic words.
- lower() converts all words to lowercase.
- ps.stem(word) applies Porter Stemming to each individual word.
- " ".join() combines the resulting stems into a processed document.
- head() displays the first five original and processed documents.

5.5 Example of Original and Stemmed Words

Original word	Porter stem	Analysis
organization	organ	Derivational information lost

organizer	organ	Derivational information lost
organizing	organ	Verb form reduced
organized	organ	Verb form reduced
organization	organ	Same stem despite different meanings
running	run	Inflectional form
studies	studi	Spelling and suffix reduction
studied	studi	Inflectional form
studying	studi	Inflectional form
happiness	happi	Derivational information lost
happily	happili	Adverb-forming suffix reduced
relational	relat	Derivational information lost
connection	connect	Noun-forming suffix removed
connected	connect	Inflectional form
connecting	connect	Inflectional form
analysis	analysi	Stem may not be a complete word
analytical	analyt	Derivational information lost
infectious	infecti	Stem may not be a complete word
infected	infect	Inflectional form
infection	infect	Noun-forming suffix removed

5.6 Analysis of 20 Stemming Cases

The examples show two main types of information loss:

- Inflectional forms: Words such as running, connected, and studied have grammatical variations. Reducing them to a common base can be useful.
- Derivational forms: Words such as organization, happiness, and infection contain suffixes that contribute meaning or change grammatical category. Removing those suffixes can erase useful distinctions.

5.7 Dataset Execution

To perform the experiment, place BBCNews.csv in the same folder as the Python program and ensure that the dataset contains the Text column. The corrected program will display the first five documents alongside their processed versions.

The dataset has not been loaded in this answer, so the displayed examples are illustrative rather than actual BBC dataset execution results.

5.8 Conclusion

The main error was applying a single-word stemming function directly to complete documents. Tokenizing the text and stemming each token fixes the coding error. However, a better linguistic strategy may be needed to prevent the loss of derivational information.

Question 6: Error Analysis of a Finite-State Morphological Parser

6.1 Given Program

```
def parser(word):
    if word.endswith("s"):
        return word[:-2], "Plural Noun"
    else:
        return word, "Singular"
words = ["cars", "boxes", "cities", "children"]
for w in words:
    print(parser(w))6.2 Identification of Logical Errors
```

6.2 Identification of Logical Errors

1. word[:-2] removes two characters instead of one, causing incorrect results for regular plurals.
2. The parser does not handle -es endings properly.
3. It does not recognize the spelling change -ies → y.
4. It cannot identify irregular plurals such as children.
5. It assumes every word ending in s is plural, which can misclassify singular nouns.

6.3 Corrected Python Program

```python
irregular = {
    "children": "child",
    "men": "man",
    "women": "woman",
    "mice": "mouse",
    "feet": "foot",
    "teeth": "tooth"
}
def parser(word):
    word = word.lower()
    if word in irregular:
        return irregular[word], "Irregular Plural"
    if word.endswith("ies") and len(word) > 3:
        return word[:-3] + "y", "Plural Noun (-ies)"
    if word.endswith(("ches", "shes", "xes", "zes", "sses")):
        return word[:-2], "Plural Noun (-es)"
    if word.endswith("s") and not word.endswith("ss"):
        return word[:-1], "Regular Plural"
    return word, "Singular or Unknown"
words = [
    "cars", "boxes", "cities", "children",
    "dogs", "watches", "books", "child"
]
print("Word -> Root -> Classification")
for word in words:
    root, category = parser(word)
    print(word, "->", root, "->", category)
```

6.4 Expected Output

```
PS C:\Users\Hasnika\OneDrive\Desktop\NLP.py> & C:\Users\Hasnika\AppDa
p/NLP.py/co2at31.py
Word -> Root -> Classification
cars -> car -> Regular Plural
boxes -> box -> Plural Noun (-es)
cities -> city -> Plural Noun (-ies)
children -> child -> Irregular Plural
dogs -> dog -> Regular Plural
watches -> watch -> Plural Noun (-es)
books -> book -> Regular Plural
child -> child -> Singular or Unknown
PS C:\Users\Hasnika\OneDrive\Desktop\NLP.py>
```

6.5 Explanation of the Corrected Parser

- Regular plurals: Removes -s, as in cars → car.
- Words ending in -es: Removes the suffix, as in boxes → box.
- Words ending in -ies: Replaces the ending with y, as in cities → city.
- Irregular plurals: Uses a dictionary, as in children → child.
- Singular words: Returns the original word when no plural rule matches.

6.6 Using WordNet or an English Lexicon

```
import nltk
nltk.download("wordnet")

from nltk.corpus import wordnet as wn

for word in ["car", "box", "city", "child"]:
    print(word, ":", bool(wn.synsets(word)))
```

6.7 Limitations of Rule-Based Parsing

- English has many irregular plural forms.

- Some words ending in s are singular.
- Spelling changes require special rules.
- The same word may have multiple grammatical roles.
- New words and social media spellings may not appear in the rule set.
- Larger rule systems require careful ordering to prevent incorrect matches.

6.8 Conclusion

The corrected parser handles common regular and irregular plural forms more accurately than the original program. Combining finite-state rules with a lexical database improves reliability, but complete morphological analysis still requires exceptions and contextual information.

Question 7: Error Analysis of Morphological Feature Extraction

7.1 Introduction

Feature extraction converts text into numerical representations that machine learning models can use. CountVectorizer creates a vocabulary of unique tokens and represents documents using token counts.

Applying stemming only after feature extraction is incorrect because the vectorizer has already created separate features for different word forms.

7.2 Problem Statement

The given program creates the vocabulary using CountVectorizer and then stems the feature names. This changes the displayed feature names but does not merge the corresponding columns in the original feature matrix.

As a result, the model may still treat related forms as separate features.

7.3 Identification of the Error

Original code:

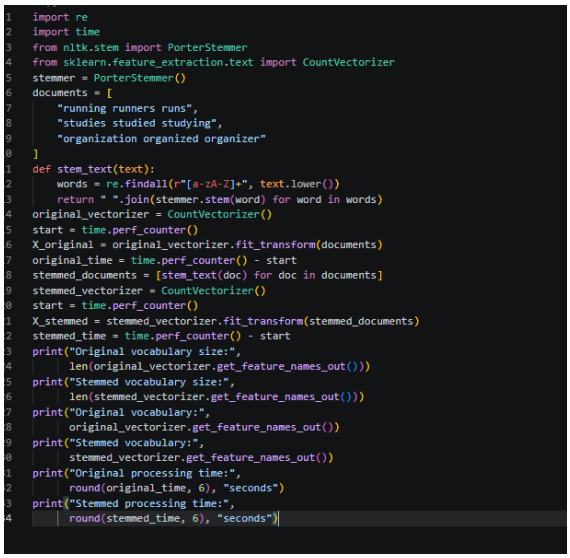
```
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(documents)


features = [
    stemmer.stem(word)
    for word in vectorizer.get_feature_names_out()
```

]

Main error: Stemming occurs after feature extraction. The vocabulary and matrix X are not rebuilt using the stemmed tokens.

7.4 Corrected Python Program



```
import re
import time
from nltk.stem import PorterStemmer
from sklearn.feature_extraction.text import CountVectorizer
stemmer = PorterStemmer()
documents = [
    "running runners runs",
    "studies studied studying",
    "organization organized organizer"
]
def stem_text(text):
    words = re.findall(r"[a-zA-Z]+", text.lower())
    return " ".join(stemmer.stem(word) for word in words)
original_vectorizer = CountVectorizer()
start = time.perf_counter()
X_original = original_vectorizer.fit_transform(documents)
original_time = time.perf_counter() - start
stemmed_documents = [stem_text(doc) for doc in documents]
stemmed_vectorizer = CountVectorizer()
start = time.perf_counter()
X_stemmed = stemmed_vectorizer.fit_transform(stemmed_documents)
stemmed_time = time.perf_counter() - start
print("Original vocabulary size:",
      len(original_vectorizer.get_feature_names_out()))
print("Stemmed vocabulary size:",
      len(stemmed_vectorizer.get_feature_names_out()))
print("Original vocabulary:",
      original_vectorizer.get_feature_names_out())
print("Stemmed vocabulary:",
      stemmed_vectorizer.get_feature_names_out())
print("Original processing time:",
      round(original_time, 6), "seconds")
print("Stemmed processing time:",
      round(stemmed_time, 6), "seconds")
```

7.5 Expected Analysis

Evaluation measure	Before correction	After correction
Stemming position	After feature extraction	Before feature extraction
Vocabulary	Contains separate word forms	Related forms may be merged
Feature matrix	Original tokens remain separate	Uses stemmed tokens
Semantic information	Original forms preserved	Some distinctions may be lost
Processing time	Usually lower preprocessing cost	Includes stemming overhead
Classification accuracy	Depends on the dataset	May improve or decrease

For the three sample documents, the original vocabulary contains nine distinct tokens. The corrected stemmed vocabulary is smaller because related forms are combined; its exact size is calculated by the program.

7.6 Evaluation Using a Real-World Dataset

The same experiment can be applied to the 20 Newsgroups dataset or another labeled text corpus. To compare classification accuracy:

1. Load the labeled dataset.
2. Split the data into training and testing sets.
3. Build one vectorizer without stemming.

4. Build another vectorizer after stemming.
5. Train the same classifier on both feature matrices.
6. Compare accuracy, vocabulary size, and processing time.

Classification accuracy can be calculated as:

$$\text{Accuracy} = \frac{\text{Correct predictions}}{\text{Total predictions}} \times 100$$

The sample program uses three documents without class labels, so it measures vocabulary size and processing time but cannot calculate meaningful classification accuracy. A labeled real-world dataset is required for that comparison.

7.7 Interpretation

Correcting the pipeline ensures that stemming takes place before vocabulary construction. This allows the vectorizer to merge matching stems into the same feature, reducing vocabulary redundancy.

However, a smaller vocabulary does not always produce better accuracy. Porter Stemming may merge words with different meanings, so performance must be evaluated experimentally.

7.8 Conclusion

The error in the original program is that stemming occurs after feature extraction. The corrected pipeline stems individual words first and then builds the vocabulary and feature matrix. This can reduce redundant features and improve model efficiency, although the final effect on classification accuracy depends on the dataset and task.